

Шпаргалка по командам Git

НАСТРОЙКА И УСТАНОВКА

Команда	Что делает
<code>git --version</code>	Проверяет установленную версию Git
<code>git config --global user.name "Имя"</code>	Задаёт имя автора для всех коммитов
<code>git config --global user.email "email"</code>	Задаёт email автора для всех коммитов
<code>git config --list</code>	Показывает все активные настройки Git

ИНИЦИАЛИЗАЦИЯ И КЛОНИРОВАНИЕ

Команда	Что делает
<code>git init</code>	Создаёт новый локальный репозиторий в текущей папке
<code>git clone <url></code>	Скачивает удалённый репозиторий на локальный компьютер

РАБОТА С ФАЙЛАМИ И СТАТУСОМ

Команда	Что делает
<code>git status</code>	Показывает текущее состояние файлов (изменённые, добавленные, неотслеживаемые)
<code>git add <файл></code>	Добавляет файл в область индексирования (Staging Area)
<code>git add .</code>	Добавляет ВСЕ изменённые и новые файлы в индекс
<code>git rm --cached <файл></code>	Удаляет файл из индекса, но оставляет его на диске

КОММИТЫ (ФИКСАЦИЯ ИЗМЕНЕНИЙ)

Команда	Что делает
<code>git commit -m "сообщение"</code>	Создаёт коммит с указанным сообщением
<code>git commit -am "сообщение"</code>	Добавляет ВСЕ отслеживаемые файлы в индекс и создаёт коммит (НОВЫЕ файлы НЕ попадают)

ИСТОРИЯ И ПРОСМОТР ЛОГОВ

Команда	Что делает
<code>git log</code>	Показывает полную историю коммитов
<code>git log --oneline</code>	Показывает историю сокращённо (один коммит — одна строка)
<code>git log --graph</code>	Показывает историю в виде дерева с визуальными связями
<code>git log --graph --oneline</code>	Компактное дерево коммитов (звёздочки — коммиты, линии — связи)

ПЕРЕКЛЮЧЕНИЕ МЕЖДУ ВЕРСИЯМИ

Команда	Что делает
<code>git checkout <хеш_коммита></code>	Переключает всю рабочую директорию на состояние указанного коммита
<code>git checkout <хеш_коммита> -- <файл></code>	Восстанавливает ТОЛЬКО указанный файл из прошлого коммита

<code>git checkout main</code>	Возвращает в актуальное состояние на вершине ветки main
--------------------------------	---

ВЕТВЛЕНИЕ (BRANCHES)

Команда	Что делает
<code>git branch</code>	Показывает список всех веток (* — текущая активная)
<code>git branch <имя_ветки></code>	Создаёт новую ветку (но не переключается на неё)
<code>git checkout -b <имя_ветки></code>	Создаёт ветку И сразу переключается на неё
<code>git switch <имя_ветки></code>	Переключается на другую ветку (современная альтернатива checkout)
<code>git branch -d <имя_ветки></code>	Удаляет ветку (только если она уже слита)
<code>git branch -D <имя_ветки></code>	Принудительно удаляет ветку (ДАЖЕ если не слита)

СЛИЯНИЕ (MERGE)

Команда	Что делает
<code>git merge <имя_ветки></code>	Сливает указанную ветку в текущую (создаёт коммит слияния)

```
git revert -m 1 <хеш_коммита_слияния>
```

Откатывает коммит слияния,
возвращая состояние до слияния

УДАЛЁННЫЕ РЕПОЗИТОРИИ (REMOTE)

Команда	Что делает
<pre>git remote add origin <url></pre>	Связывает локальный проект с удалённым репозиторием (псевдоним origin)
<pre>git push origin <ветка></pre>	Отправляет локальные коммиты в удалённый репозиторий
<pre>git pull origin <ветка></pre>	Скачивает свежие коммиты с сервера И сразу сливает их с локальной веткой
<pre>git fetch origin</pre>	Скачивает информацию об изменениях на сервере, НО НЕ сливает их

ИГНОРИРОВАНИЕ ФАЙЛОВ (.gitignore)

Команда	Что делает
Создать файл <pre>.gitignore</pre>	Добавляет правила игнорирования файлов и папок (построчно)

РАБОТА С КОНФЛИКТАМИ

Маркеры конфликта в файле:

Маркер	Значение
<<<<<< HEAD	Начало изменений из текущей ветки
=====	Разделитель между конфликтующими версиями
>>>>>> <ветка>	Конец изменений из сливаемой ветки

Действия после разрешения конфликта:

Команда	Что делает
git add <файл>	Индексирует исправленный файл после ручного разрешения конфликта
git commit -m "сообщение"	Фиксирует разрешение конфликта

ПОЛНЫЙ РАБОЧИЙ ЦИКЛ (ВКРАТЦЕ)

git status	# Проверить состояние
git add .	# Добавить всё в индекс
git commit -m "сообщение"	# Создать коммит
git log --oneline	# Посмотреть историю

<code>git checkout -b новая_ветка</code>	# Создать и переключиться на ветку
<code>git switch main</code>	# Вернуться на main
<code>git merge новая_ветка</code>	# Слить ветку в main
<code>git push origin main</code>	# Отправить на GitHub
<code>git pull origin main</code>	# Скачать изменения с сервера